

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0624	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	S.MAGESH	Reg. No.:	192525002
Section / Batch:	B. TECH AIML	Date:	19/07/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems.
- Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required.
- Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis.
- Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1

SECTION A – APPLICATION BASED QUESTIONS

Q1. Selection Sort – Analytical Problem Apply the Brute Force technique using Selection Sort on the dataset [64, 25, 12, 22, 11]. Clearly interpret the problem and justify why Selection Sort is suitable. Perform the sorting step-by-step and show intermediate results after each pass. Count the total number of comparisons made during execution. Analyze the time complexity in best, average, and worst cases. Verify the correctness of the final sorted output. Interpret the efficiency of the algorithm based on the results.

1. Understanding the Problem

The given dataset is [64, 25, 12, 22, 11]. The objective is to sort the elements in ascending order using the Selection Sort algorithm. Selection Sort repeatedly selects the smallest element from the unsorted portion of the array and places it in the correct position. It is simple to understand and suitable for small datasets.

2. Step-by-Step Solution

Initial Array

64 25 12 22 11

Pass 1

- Smallest element = 11
- Swap 64 and 11

Array after Pass 1:

11 25 12 22 64

Pass 2

- Smallest element = 12
- Swap 25 and 12

Array after Pass 2:

11 12 25 22 64

Pass 3

- Smallest element = 22

- Swap 25 and 22

Array after Pass 3:

11 12 22 25 64

Pass 4

- Remaining elements are already in order.

Final Array:

11 12 22 25 64

Number of Comparisons

- Pass 1 = 4
- Pass 2 = 3
- Pass 3 = 2
- Pass 4 = 1

Total Comparisons = 10

Time Complexity

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Conclusion

Selection Sort successfully arranged the elements in ascending order. Although it is easy to implement, it performs the same number of comparisons in all cases, making it less efficient for large datasets.

Q4. Searching Techniques – Comparative Analysis Analyze the performance of Linear Search on unsorted data and Binary Search on sorted data. Interpret the problem and explain the working of both algorithms. Apply both methods and count the number of comparisons required. Analyze their time complexities in different cases. Compare their efficiency based on dataset conditions. Justify when each method is preferable. Interpret the results in terms of real-world applications.

1. Understanding the Problem

The objective is to compare the performance of Linear Search and Binary Search. Linear Search checks every element one after another, whereas Binary Search repeatedly divides a sorted array into two halves to locate the required element.

Linear Search

Dataset:

23 45 12 67 34

Search Element:

34

Comparison	Element Checked	Result
1	23	Not Found
2	45	Not Found
3	12	Not Found
4	67	Not Found
5	34	Found

Total Comparisons = 5

Binary Search

Sorted Dataset:

12 23 34 45 67

Step	Middle Element	Result
1	34	Found

Total Comparisons = 1

Algorithm	Best	Average	Worst
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)

Conclusion

Linear Search is suitable for unsorted datasets, while Binary Search is much faster for sorted datasets because it reduces the search space after every comparison.

Q6. Bubble Sort – Analytical Problem Apply the Brute Force technique using Bubble Sort on the dataset [45, 12, 78, 34, 23]. Clearly interpret the problem and justify why Bubble Sort is used. Perform the sorting step-by-step and show intermediate results after each pass. Count the total number of comparisons and swaps made. Analyze the time complexity in best, average, and worst cases. Verify the correctness of the final sorted output. Interpret the efficiency of the algorithm.

1. Understanding the Problem

The dataset [45, 12, 78, 34, 23] needs to be sorted in ascending order using Bubble Sort. Bubble Sort compares adjacent elements and swaps them whenever they are in the wrong order.

2. Step-by-Step Solution

Initial Array

45 12 78 34 23

Pass 1

12 45 34 23 78

Pass 2

12 34 23 45 78

Pass 3

12 23 34 45 78

Pass 4

12 23 34 45 78

Number of Comparisons

- Pass 1 = 4
- Pass 2 = 3
- Pass 3 = 2
- Pass 4 = 1

Total Comparisons = 10

Number of Swaps

Total Swaps = 5

Time Complexity

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Final Output

12 23 34 45 78

Conclusion

Bubble Sort correctly sorted the dataset. It is simple to implement but performs many comparisons and swaps, making it less suitable for large datasets.

Q7. Linear Search – Analytical Problem Given an unsorted dataset [23, 45, 12, 67, 34, 89, 10], apply Linear Search to find the element 34. Clearly interpret the problem and explain the search process step-by-step. Count the number of comparisons required to locate the element. Analyze the time complexity in best, average, and worst cases. Verify the correctness of the result. Interpret the efficiency of Linear Search in real-world scenarios.

1. Understanding the Problem

The given dataset is [23, 45, 12, 67, 34, 89, 10]. The objective is to find the element **34** using Linear Search. The algorithm checks each element sequentially until the target is located.

2. Step-by-Step Search

Comparison	Element Checked	Result
1	23	Not Found
2	45	Not Found
3	12	Not Found
4	67	Not Found
5	34	Found

Total Comparisons

5

Time Complexity

- Best Case: **$O(1)$**
- Average Case: **$O(n)$**
- Worst Case: **$O(n)$**

Result

The element **34** is found at the **5th position**.

Conclusion

Linear Search successfully located the required element after five comparisons. It is easy to implement and works well for small or unsorted datasets.

Q8. String Matching – Brute Force Analysis Given a text “ABABABCABABABCAB” and a pattern “ABABC”, apply the Brute Force string matching algorithm. Clearly interpret the problem and justify the method selection. Compare the pattern with the text at each position and show all matching steps. Count the number of comparisons performed. Identify all occurrences of the pattern. Analyze the time complexity of the approach. Interpret its efficiency compared to optimized methods.

1. Understanding the Problem

The given text is:

ABABABCABABABCAB

The pattern is:

ABABC

The Brute Force String Matching algorithm compares the pattern with every possible position in the text until all matches are identified.

2. Matching Process

Position	Result
1	No Match
2	No Match
3	Match Found
4	No Match
5	No Match
6	No Match
7	No Match
8	No Match
9	Match Found

Pattern Occurrences

- Position 3
- Position 9

Number of Comparisons

Approximately 25 comparisons are required.

Time Complexity

- Best Case: $O(n)$
- Worst Case: $O(n \times m)$

where:

- n = Length of the text
- m = Length of the pattern

Conclusion

The Brute Force String Matching algorithm successfully found all occurrences of the pattern. It is easy to understand and implement but performs many unnecessary comparisons, making it less efficient than optimized algorithms such as KMP for larger datasets.

Code of Conduct

*I **S.MAGESH(192525002)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:



RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____